

15강

this 키워드 & static

및

정보은닉



this 키워드





➤ this 란?

- 객체 외부에서 인스턴스 멤버에 접근하기 위해 참조 변수를 사용하는 것과 마찬가지로 **객체 내부에서도 인스턴스 멤버에 접근하기 위해 this 키워드를 사용할 수 있다.** 우리가 자신을 '나' 라고 가리키듯 객체는 자신을 this라고 한다. 따라서 this.name은 자신이 가지고 있는 name필드(속성)라는 뜻이다. **this는 주로 생성자와 메소드의 매개 변수 이름이 필드와 동일한 경우, 인스턴스 멤버인 필드를 명시하고자 할 때 사용한다.**

```
1 package testclass;
2
3
4 class Fruit {
5
6     // 멤버변수
7     String name;
8     String color;
9     double weight;
10    int count;
11
12    // 생성자
13    public Fruit(String name, String color, double weight, int count) {
14        name = name;
15        color = color;
16        weight = weight;
17        count = count;
18    }
19 }
20
21 public class Test10 {
22
23     public static void main(String[] args) {
24         Fruit fir = new Fruit("banana", "yellow", 5.0, 10);
25         System.out.println("name : " + fir.name);
26         System.out.println("color : " + fir.color);
27         System.out.println("weight : " + fir.weight);
28         System.out.println("count : " + fir.count);
29     }
30 }
31
32 }
```

전역변수 : Fruit 객체 전 지역에서 사용되는 변수

지역변수 : Fruit 생성자 지역에서만 사용되는 변수

name : null
color : null
weight : 0.0
count : 0

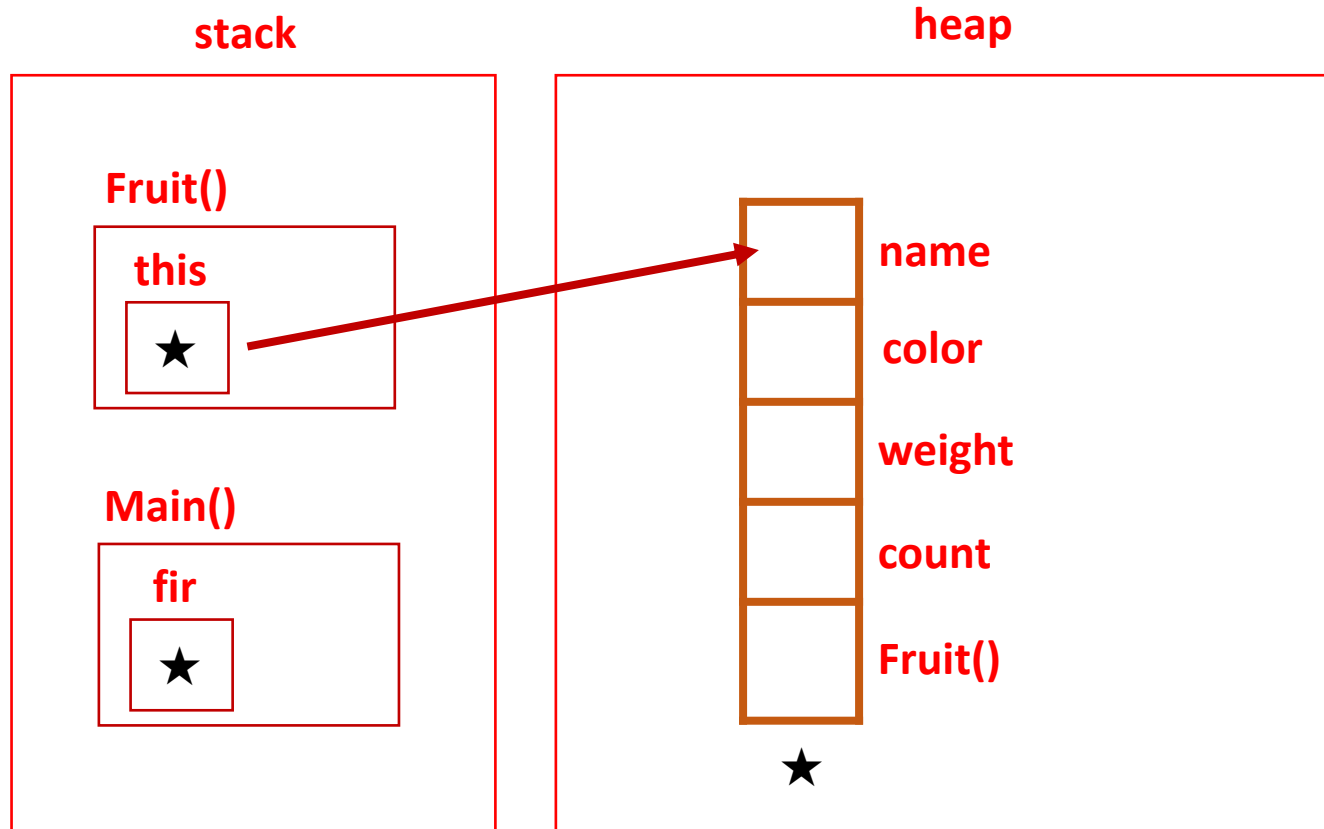


- 클래스의 속성(멤버변수)과 생성자 / 메서드의 매개변수(input parameter)의 이름이 같은 경우
- 클래스 속성을 사용할 때 **this** 키워드를 붙여줍니다.
 - 여기서 **this** 키워드의 의미는 **Fruit** 객체 자기자신을 의미, 즉 Fruit 의 전역 변수를 의미한다.

```
1
2
3 package testclass;
4
5 class Fruit{
6     String name;
7     String color;
8     double weight;
9     int count;
10 // 생성자
11 public Fruit(String name, String color, double weight, int count) {
12     // 여기서 this키워드는 Fruit 객체 자기자신을 의미 한다.
13     this.name = name;
14     this.color = color;
15     this.weight = weight;
16     this.count = count;
17 }
18 }
19 }
20
21 public class Test10 {
22
23     public static void main(String[] args) {
24         Fruit fir = new Fruit("banana","yellow",5.0,10);
25         System.out.println("name : "+fir.name);
26         System.out.println("color : "+fir.color);
27         System.out.println("weight : "+fir.weight);
28         System.out.println("count : "+fir.count);
29
30     }
31
32 }
```



```
name : banana
color : yellow
weight : 5.0
count : 10
|
```



static





➤ static 란?

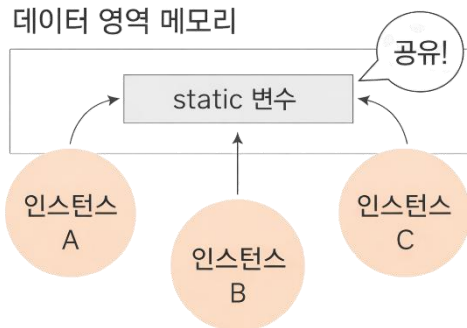
- 정적 필드와 정적 메소드를 선언 하려면 필드와 메소드 선언 시 static를 추가적 으로 붙이면 된다. 다음은 정적 멤버와 정적 메소들 를 선언하는 방법이다.

```
Public class 클래스 이름{  
  
    //정적 필드  
    static 타입 필드 [=초기값];  
  
    //정적 메소드  
    static 리턴 타입 메소드 이름( 매개변수 선언....){.....}  
  
}
```

- 정적 필드와 정적 메소드는 클래스에 고정된 멤버 이므로 클래스 로더가 클래스(바이트 코드)를 로딩해서 메소드 메모리 영역에 적재할 때 클래스별로 관리된다. 따라서 클래스의 로딩(load)이 끝나면 바로 사용할 수 있다.
- 여러 개의 인스턴스가 **같은 메모리의 값을 공유**하기 위해 사용



데이터 영역 메모리





➤ static 변수

- Static 변수는 인스턴스가 생성될 때 마다 다른 메모리를 가지는 것이 아니라 프로그램이 메모리에 적재(load)될 때 데이터 영역의 메모리에 생성 됨
- 따라서 인스턴스의 생성과 관계없이 클래스 이름으로 직접 참조함

Student.serialNum = 100; // serialNum이 static 변수

클래스 변수라고도 함

- 멤버변수는 다른 말로 인스턴스 변수라고 함

[Student] 클래스 생성

```
package staticTest;

public class Student {
    //학생들의 학번을 1001, 1002, 1003으로 출력하려고 할때
    // 기본 serialNum에 1000으로 정적 멤버변수를 이용하여
    // 값을 지정하면 여러 개의 인스턴스가 같은 serialNum를 공유할 수 있다.
    static int serialNum = 1000;

    int studentId;
    String studentName;

    //생성자
    public Student() {
        serialNum++;
        studentId = serialNum;
    }
}
```




[StudentTest1] 클래스 생성

```
public class StudentTest1 {  
    public static void main(String[] args) {  
        //인스턴스화  
        Student studentJ = new Student();  
        System.out.println(studentJ.serialNum);  
        studentJ.serialNum++;  
  
        Student studentT = new Student();  
        System.out.println(studentJ.serialNum);  
        System.out.println(studentT.serialNum);  
    }  
}
```

```
Console X  
<terminated> Stu  
1000  
1001  
1001
```

- 객체화(=인스턴스)화 한 상태에서 static(=정적)으로 작성한 정적 멤버 변수인 serialNum을 StudentTest1 클래스에서 1씩 누적하면 각각의 객체인 studentJ와 studentT의 모두 같은 값이 출력된다.
- 이유는 Student클래스의 static 멤버인 serialNum의 값을 모두 공유하기 때문이다.이런 경우는 Student 클래스에 아래와 같은 생성자를 추가한다.

- //생성자

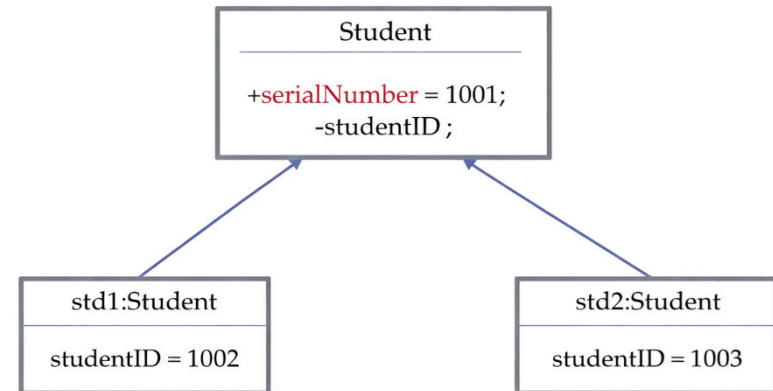
```
public Student() {  
    serialNum++;  
    studentId = serialNum;  
}
```



[StudentTest1] 클래스 생성

```
public class StudentTest1 {  
    public static void main(String[] args) {  
        //인스턴스화  
        Student studentJ = new Student();  
        System.out.println(studentJ.studentId);  
  
        Student studentT = new Student();  
        System.out.println(studentT.studentId);  
  
        System.out.println(studentJ.serialNum);  
        System.out.println(studentT.serialNum);  
    }  
}
```

```
Console X  
<terminated> Stu  
1001  
1002  
1002  
1002
```

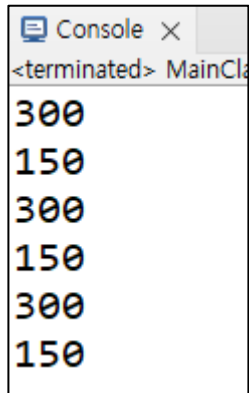


note : serialNumber를 static 으로 선언하면 모든 student instance에 대해 하나의 변수로 유지되고 이러한 변수를 class 변수라 한다.



- **static이 붙은 멤버 변수는 인스턴스를 생성하지 않아도 사용할 수 있다.**

```
public class Card {  
  
    static int width  = 150;  
    static int height = 300;  
  
    int number;  
    String shape;  
  
}
```



```
Console x  
<terminated> MainClass  
300  
150  
300  
150  
300  
150
```

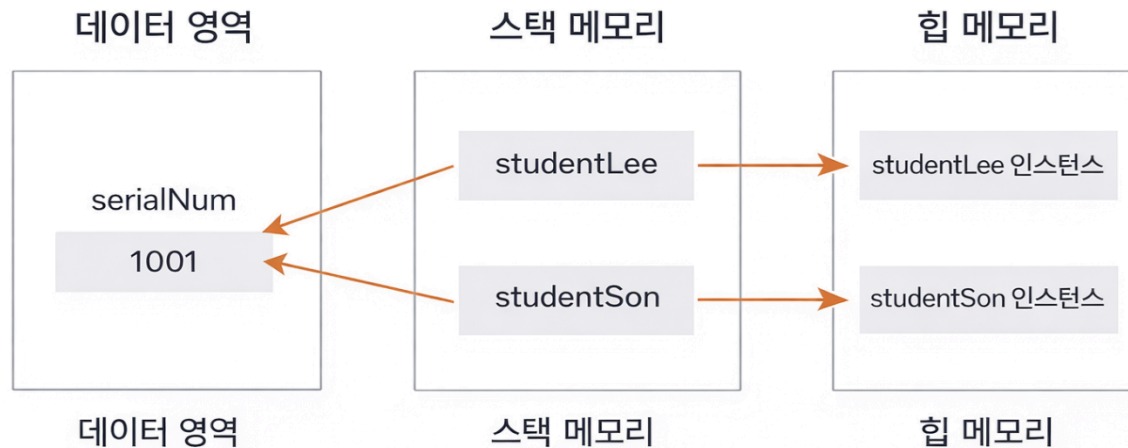
```
public class MainClass {  
  
    public static void main(String[] args) {  
  
        Card c1 = new Card();  
        c1.number = 4;  
        c1.shape = "heart";  
  
        Card c2 = new Card();  
        c2.number = 7;  
        c2.shape = "spade";  
  
        c1.width = 300;  
        c1.height = 150;  
  
        System.out.println(Card.width); // 300  
        System.out.println(Card.height); // 150  
  
        System.out.println(c1.width); // 300  
        System.out.println(c1.height); // 150  
  
        System.out.println(c2.width); // 300  
        System.out.println(c2.height); // 150  
    }  
  
}
```



➤ static 변수 예

- 여러 인스턴스가 하나의 메모리 값을 공유 할 때 필요
- 학생이 생성될 때마다 학번이 증가해야 하는 경우
- 기준이 되는 값은 static 변수로 생성하여 유지 함

각 학생이 생성될 때 마다 static 변수 값을 복사해 와서 하나 증가 시킨 값을 생성된 인스턴스의 학번 변수에 저장해 준다.





➤ static 사용 이유

1. 클래스를 설계할 때, 멤버변수 중 모든 인스턴스에 공통적으로 사용해야하는 것에 static을 붙인다.

- 인스턴스를 생성하면, 각 인스턴스들은 서로 독립적이기 때문에 서로 다른 값을 유지한다. 경우에 따라서는 각 인스턴스들이 공통적으로 같은 값이 유지되어야 하는 경우 static을 붙인다.

2. static이 붙은 멤버변수는 인스턴스를 생성하지 않아도 사용할 수 있다.

- static이 붙은 멤버변수(클래스변수)는 클래스가 메모리에 올라갈때 이미 자동적으로 생성되기 때문이다.

3. static이 붙은 메서드(함수)에서는 인스턴스 변수를 사용할 수 없다.

- static 메서드는 인스턴스 생성 없이 호출가능한 반면, 인스턴스 변수는 인스턴스를 생성해야만 존재하기 때문에... static이 붙은 메서드(클래스메서드)를 호출할 때 인스턴스가 생성되어있을수도 그렇지 않을 수도 있어서 static이 붙은 메서드에서 인스턴스변수의 사용을 허용하지 않는다. (반대로, 인스턴스변수나 인스턴스메서드에서는 static이 붙은 멤버들을 사용하는 것이 언제나 가능하다. 인스턴스변수가 존재한다는 것은 static이 붙은 변수가 이미 메모리에 존재한다는 것을 의미하기 때문이다.)

4. 메서드 내에서 인스턴스 변수를 사용하지 않는다면, static을 붙이는 것을 고려한다.

- 메서드의 작업내용중에서 인스턴스 변수를 필요로 한다면, static을 붙일 수 없다. 반대로 인스턴스변수를 필요로 하지 않는다면, 가능하면 static을 붙이는 것이 좋다. 메서드 호출시간이 짧아지기 때문에 효율이 높아진다. (static을 안붙인 메서드는 실행시 호출되어야할 메서드를 찾는 과정이 추가적으로 필요하기 때문에 시간이 더걸린다.)



➤ 변수의 유효 범위

변수 유형	선언 위치	사용 범위	메모리	생성과 소멸
지역 변수 (로컬 변수)	함수(메서드) 내부에 선언	해당 함수 내부에서만 사용	스택(Stack)	함수가 호출될 때 생성되고 함수가 종료되면 소멸
멤버 변수 (인스턴스 변수)	클래스 내부에 선언	클래스 내부에서 사용 private이 아니면 참조 변수로 다른 클래스에서 사용 가능	힙(Heap)	인스턴스가 생성될 때 힙에 생성되고가비지 컬렉터가 메모리를 수거할 때 소멸
static 변수 (클래스 변수)	클래스 내부에서static 키워드로 선언	클래스 내부에서 사용 private이 아니면 클래스 이름으로 다른 클래스에서 사용 가능	데이터 영역 (Method Area) • 클래스 정보 (클래스 구조, 메서드 정보) • static 변수 (클래스 변수) • static 메서드 • 상수(Constant Pool) • String Pool (문자열 상수 풀)	프로그램 시작 시 데이터 영역에 생성되고 프로그램 종료 시 소멸



➤ 여기서 잠깐 !

- staticExample 패키지안에 StaticEx01.java, MainClass.java class 파일을 각각 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

◆ 작성 조건

- static 정적 변수를 이용하여 프로그램을 작성하시오.
- StaticEx01 클래스에는
- 멤버 변수, 생성자, 메서드를 자유롭게 작성하시오.
- ATM 잔액은 static 변수를 이용해 관리하시오.

◆ 기능 요구사항

입금 기능

- 사용자의 이름과 입금 금액을 전달받아 ATM 잔액을 증가시킨다.
- 입금 후 아래 형식으로 출력한다.

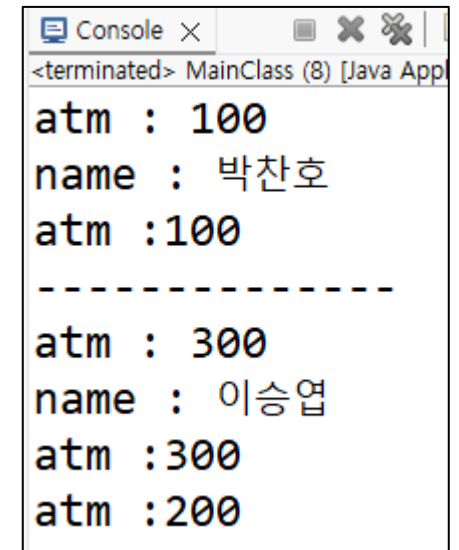
출금(또는 이체) 기능

- 출금(또는 이체) 금액을 전달받아 ATM 잔액을 감소시킨다.
- 출금(또는 이체) 후 현재 잔액을 출력한다.

◆ 실행 조건 (출력 결과 기준)

- 박찬호의 ATM에 100원 입금하면
- 이승엽의 ATM에 300원 입금하면
- 이후 이승엽의 ATM에서 100원을 출금(또는 이체) 하면 <출력 결과>와 같이 출력되도록 작성하시오.

<출력 결과>



```
Console x
<terminated> MainClass (8) [Java Appl
atm : 100
name : 박찬호
atm :100
-----
atm : 300
name : 이승엽
atm :300
atm :200
```

패키지 및 접근 제한자



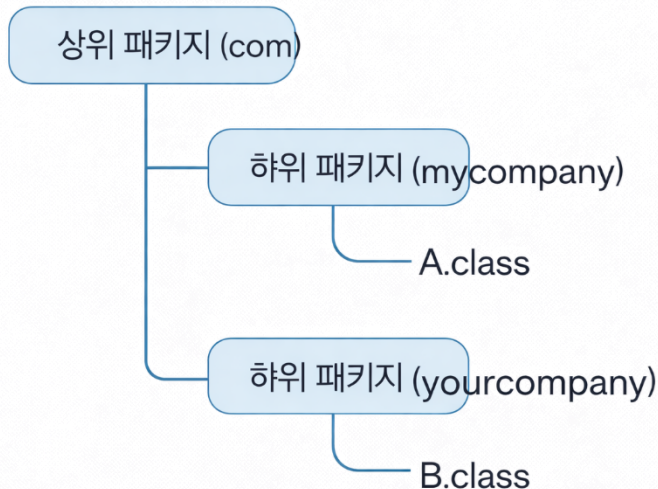


➤ 패키지(Package) 란 ?

- 패키지의 물리적인 형태는 파일 시스템의 **폴더**이다. 패키지는 단순히 파일 시스템의 폴더 기능만 하는 것이 아니라 클래스의 일부분으로 클래스를 **유일하게** 만들어주는 **식별자** 역할을 한다.
- 클래스 이름이 동일하더라도 패키지가 다르면 다른 클래스로 인식한다.
- 클래스의 전체 이름은 패키지 이름 + 클래스 이름 인데 패키지가 상, 하위로 구분되어 있다면 도트(.)을 사용해 다음과 같이 표현한다.

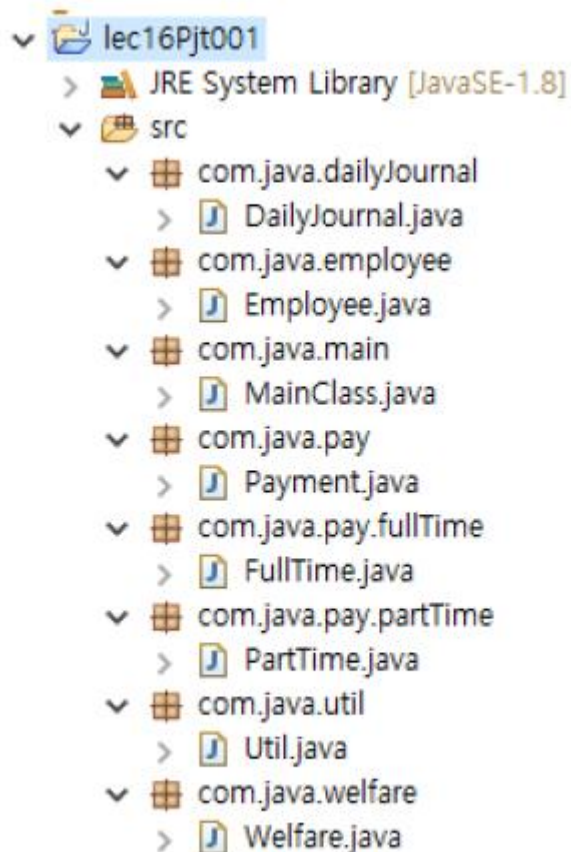
상위패키지.하위패키지.클래스

- 예를 들어 아래와 같이 패키지가 구성되어 있다면, A 클래스의 전체 이름은 com.mycompany.A이고, B클래스의 전체 이름은 com.mycompany.B이다.





- 다른 패키지에 있는 클래스를 사용하기 위해서는 import 키워드를 이용한다.



```
package com.java.main;
```

```
import com.java.dailyJournal.DailyJournal;  
import com.java.employee.Employee;  
import com.java.pay.Payment;  
import com.java.pay.fullTime.FullTime;  
import com.java.util.Util;  
import com.java.welfare.Welfare;
```

```
public class MainClass {
```

```
    public static void main(String[] args) {
```

```
        DailyJournal dailyJournal = new DailyJournal();
```

```
        Employee employee = new Employee();
```

```
        Payment payment = new Payment();
```

```
        FullTime fullTime = new FullTime();
```

```
        com.java.pay.partTime.PartTime partTime = new com.java.pay.partTime.PartTime();
```

```
        Util util = new Util();
```

```
        Welfare welfare = new Welfare();
```

```
    }
```

```
}
```



➤ 접근 제한자 종류

-  **public** : 모든 접근을 허용
-  **protected** : 같은 패키지(폴더)에 있는 객체와 상속관계의 객체들만 허용
(단, protected는 상속 전용 public이 아니다)
-  **default** : 같은 패키지(폴더)에 있는 객체들만 허용
(단, default는 접근 제어자를 아예 쓰지 않은 상태를 의미한다)
-  **private** : 현재 객체(클래스) 내에서만 허용

접근 제어자	외부 패키지에서 사용	상속 관계에서 사용	외부 클래스에서 사용	같은 클래스에서 사용
Public	○	○	○	○
Protected	X	○	○	○
Default	X	X	○	○
private	X	X	X	○

- ✓ 접근 범위 크기 순서
public > protected > default > private



➤ Private 접근 제한자

- 클래스의 외부에서 클래스 내부의 멤버 변수나 메서드에 접근(access)하지 못하게 하는 경우 사용한다.
- 멤버 변수나 메서드를 외부에서 직접 접근하지 못하도록 하여 제어된 방법(getter/setter)으로만 접근하게 함으로써 오류를 줄이고 객체의 상태를 보호할 수 있다.
- 변수에 대해서는 필요한 경우 `get()`, `set()` 메서드를 제공한다.

가제어/수정하기

➤ 정보 은닉(Information Hiding): 내부 데이터를 외부에서 직접 접근하지 못하게 숨기는 것

- private 접근 제한자는 클래스 외부에서 클래스 내부의 멤버 변수나 메서드에 직접 접근하지 못하게 하기 위해 사용한다.
- 이를 통해 객체의 내부 상태를 보호하고 오류를 줄일 수 있다.
- 멤버 변수의 경우, 필요한 경우에만 `get()`, `set()` 메서드를 제공하여 제한된 방식으로 접근하도록 한다.

$a = 10 \leftarrow \times$ 불가

`a.set(10)` ← 이렇게야

➤ 접근 제한자 없는 멤버 변수의 문제

- 잘못된 값도 막을 수 없음
- 누가, 언제, 왜 바꿨는지 추적 어려움
- 클래스가 자기 상태를 스스로 통제하지 못함
- 멤버 변수는 private로 숨긴다



➤ 캡슐화 (Encapsulation) 란 ?

- private을 이용한 정보 은닉은 캡슐화의 “수단(방법)” 중 하나이다.
- 객체의 데이터 + 그 데이터를 다루는 방법을 하나로 묶고 외부에서는 필요한 사용 방법만 공개하는 것 핵심 키워드는 “어떻게 구현됐는지는 숨기고, 어떻게 쓰는지만 보여준다”



➤ Public 으로 선언한 경우

```
public class Birthday {  
    // 디폴트 접근 제어자 : 접근제어자를 입력하지 않는다.(같은 패키지안에서만 사용)  
    // public : 모두 사용가능하다.  
    public int day;  
    public int month;  
    public int year;  
}
```

```
public class BirthdayTest {  
  
    public static void main(String[] args) {  
  
        Birthday bir = new Birthday();  
        // 2월에 30일이 존재하지 않는데 오류를 막을 수 없음  
        bir.month = 2;  
        bir.year = 2023;  
        bir.day = 30;  
    }  
}
```



➤ private 으로 선언한 경우

```
public class Birthday {  
    // private : 값을 숨긴다.  
    private int day;  
    private int month;  
    private int year;  
  
    // getter():출력, setter():입력 로 public 메서드를 open해준다.  
    public int getDay() {  
        return day;  
    }  
  
    public void setDay(int day) {  
        if(month == 2) {  
            if(day < 1 || day > 28) {  
                System.out.println("날짜오류");  
            }  
        } else {  
            this.day = day;  
        }  
    }  
  
    public int getMonth() {  
        return month;  
    }  
  
    public void setMonth(int month) {  
        this.month = month;  
    }  
  
    public int getYear() {  
        return year;  
    }  
  
    public void setYear(int year) {  
        this.year = year;  
    }  
}
```

날짜가 변수에 저장되기 전에 체크하여
오류를 방지 할 수 있음

```
public class BirthdayTest {  
  
    public static void main(String[] args) {  
  
        Birthday bir = new Birthday();  
        // 2월달은 28일까지이므로 이런경우 코드오류는 아니지만 로직오류가 된다.  
        bir.setMonth(2);  
        bir.setYear(2023);  
        bir.setDay(30);  
    }  
}
```



Console ×
<terminated> BirthdayTest (1)
날짜오류



➤ 여기서 잠깐 !

- **privateExample** 패키지안에 **privateEx01.java**, **MainClass.java** class 파일을 각각 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

◆ PrivateEx01 클래스

- 멤버 변수, 생성자, 메서드를 자유롭게 작성하시오.
- 멤버 변수는 반드시 **private** 접근 제한자를 사용하여 선언하시오.
- 멤버 변수는 클래스 외부에서 직접 접근할 수 없어야 한다.

◆ 기능 요구사항

값 설정 기능

- 정수 값을 전달받아 멤버 변수에 저장하는 기능을 구현하시오.

값 출력 기능

- 현재 저장된 값을 출력하는 기능을 구현하시오.

◆ 실행 조건 (출력 결과 기준)

- MainClass에서 객체를 생성한다.
- 멤버 변수에 직접 접근하려고 하면 컴파일 오류가 발생해야 한다.
- 메서드를 통해 값을 설정하고 출력했을 때 <출력 결과>와 같이 출력되도록 작성하시오.

<출력 결과>

